THE “HELLO WORLD” BARE-METAL OS

DHARUN S V (241118)

KANISHWAR S (241123)

VIGNESH R S (241159)

GITHUB LINK: https://github.com/jkrsubzer0-0/OS_Project

OVERVIEW

- WHY “HELLO WORLD” BARE-METAL OS ?
- INITIAL ATTEMPT
- PROBLEMS FACED
- RESULT OF INITIAL ATTEMPT
- CHANGES AND… WORKED
- FINAL PROJECT
- OVERALL WORK FLOW
- RESEARCH PAPERS REFERRED

WHY “HELLO WORLD” BARE-METAL OS ?

- Basic understanding on how the OS interacts with hardware.
- Ground frame for developing modern OS.
- Working of bootloader
- Combining assembly and C in low-level environment
- How to execute the OS

INITIAL ATTEMPT

- boot.s
- kernal.c
- linker.ld
- Makefile

INITIAL ATTEMPTS CONTINUED…

1. boot.s (THE ENTRY POINT)

- When a computer starts, it doesn't know how to run C code, it only knows how to execute machine instructions.
- **Role:** this is written in **assembly**. It handles the very first instructions the CPU executes.

INITIAL ATTEMPTS CONTINUED…

2. KERNEL.C (THE LOGIC)

- This is where the actual "OS" lives.
- **Role:** since no printf (because there is no os to provide it), directly to the screen's memory “Hello World has to be written”.
- Treats the screen like a grid of characters and writes

INITIAL ATTEMPTS CONTINUED…

3. Linker.ld (THE MAP)

- **Role:** it tells the compiler exactly **where** in the computer's RAM the code should be placed as the os resides at a specific physical memory.
- **Function:** it stitches the compiled boot.O and kernel.O together into a single file, ensuring the boot assembly comes first.

4. MAKEFILE (THE CONDUCTOR)

- **ROLE:** AUTOMATION.

PROBLEM FACED

- 3 MAIN PROBLEMS
- BIOS DIDN’T EXECUTE KERNEL
- KERNEL WRONG FORMAT
- WRONG MEMORY ADDRESS

BIOS DIDN’T EXECUTE KERNEL

- WHEN THE COMPUTER (OR QEMU) STARTS, THE **BIOS DID NOT LOAD THE KERNEL AUTOMATICALLY**
- BIOS OPERATES IN **16 BITS** WHEN SYSTEM STARTS BUT **32 BITS ASSEMBLY** WAS USED
- THE KERNAL WAS STORED SOMEWHERE ELSE

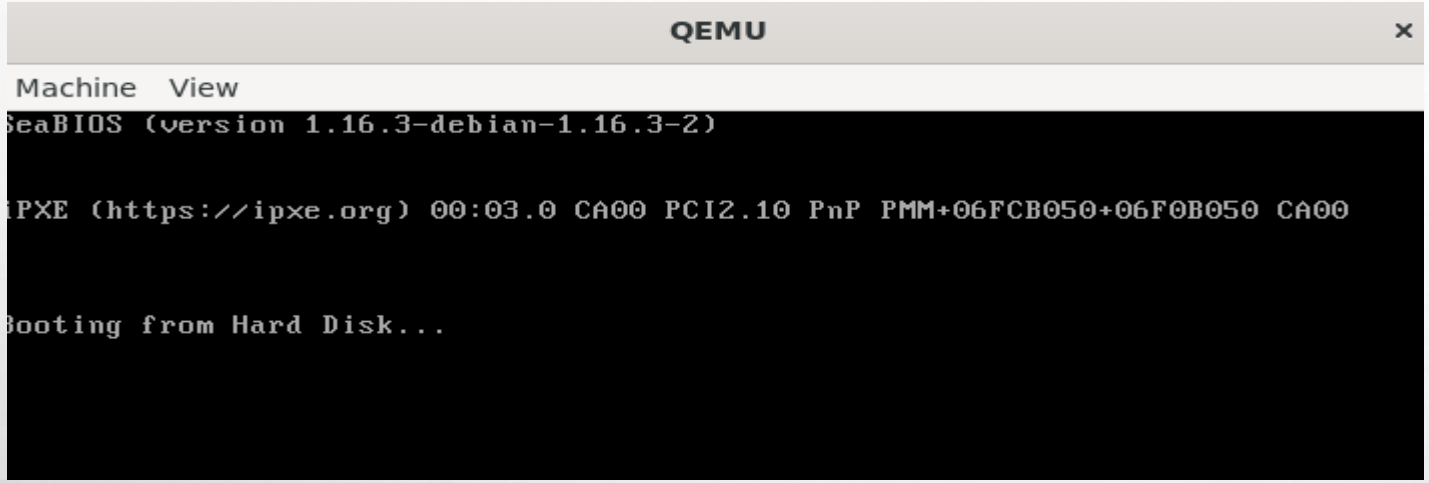
KERNEL WRONG FORMAT

- PRODUCED AN **ELF FILE,** NOT A RAW BINARY
- BIOS DOES NOT UNDERSTAND ELF (EXECUTABLE FORMAT USED BY LINUX)
- IT EXPECTS **RAW MACHINE CODE**

WRONG MEMORY ADDRESS

- KERNEL WAS COMPILED ASSUMING: . = 1M (0X100000)
- OS WAS LOADED AT 0X1000
- SO CPU JUMPED TO: 0X1000 → (BUT CODE EXPECTED AT 0X100000)

RESULT OF INITIAL ATTEMPTS:



- Wrong instruction executed
- Blank screen

CHANGES AND… WORKED

- The boot.asm file was modified to function as a valid bootloader.
- It was converted into 16-bit assembly using bits 16 (initially 32 bit)
- Most importantly, a boot signature 0xaa55 was added at the end of the file, which is required for the BIOS to recognize the code as a bootable program.
- Now, the makefile was updated to assemble the bootloader using NASM (previously elf file) with the -f bin option, which generates a flat binary file.

FINAL PROJECT

- boot.asm
- kernal.c
- makefile

BOOT.ASM

BITS 16

→ TELLS THE ASSEMBLER TO GENERATE **16-BIT CODE** (REAL MODE).

→ THIS IS REQUIRED BECAUSE THE BIOS LOADS BOOTLOADERS IN **16-BIT REAL MODE**.

ORG 0X7C00

→ SPECIFIES THAT THE CODE WILL BE LOADED AT MEMORY ADDRESS **0X7C00**.

→ THIS IS THE STANDARD ADDRESS WHERE BIOS LOADS THE BOOT SECTOR.

BOOT.ASM CONTINUED…

START:

→ Label marking the beginning of execution.

Mov si, msg

→ LOADS THE ADDRESS OF THE STRING MSG INTO THE **SI REGISTER**.

→ SI WILL BE USED TO READ CHARACTERS ONE BY ONE.

BOOT.ASM CONTINUED…

Print:

Lodsb

Print: label for loop.

Lodsb

→ Loads a byte from memory at address **[SI]** into **AL.**

→ Automatically increments SI (points to next character).

BOOT.ASM CONTINUED…

cmp al, 0

- Compare the character in AL with 0.
- This checks if we reached the end of the string. (0 acts as null terminator)

je done

→ IF AL == 0 → JUMP TO DONE

→ MEANS STRING PRINTING IS FINISHED.

BOOT.ASM CONTINUED…

- **MOV AH, 0X0E**
- SETS BIOS FUNCTION 0X0E (OUTPUT) IN REGISTER AH.
- THIS FUNCTION PRINTS A CHARACTER ON SCREEN.

INT 0X10

- CALLS **BIOS INTERRUPT 0X10** (VIDEO SERVICES).
- IT PRINTS THE CHARACTER STORED IN **AL**.

BOOT.ASM CONTINUED…

DONE:

jmp $

- KEEPS CPU HERE FOREVER

DW 0XAA55

- REQUIRED **BOOT SIGNATURE.**
- BIOS CHECKS LAST 2 BYTES:
- MUST BE **0X55AA** (LITTLE-ENDIAN → 0XAA55)
- WITHOUT THIS, SYSTEM WON'T BOOT.

KERNAL.C

char *video = (char*)0XB8000;

- 0XB8000 IS THE **VGA TEXT MODE VIDEO MEMORY ADDRESS**.
- THE SCREEN IS DIRECTLY MAPPED TO THIS MEMORY.
- EACH CHARACTER ON SCREEN USES **2 BYTES**:
- 1ST BYTE → ASCII CHARACTER
- 2ND BYTE → COLOR ATTRIBUTE (WRITING TO THIS MEMORY = DIRECTLY WRITING TO SCREEN)

KERNAL.C

char *msg = "HELLO WORLD";

- STORES THE STRING YOU WANT TO DISPLAY.
- THIS STRING IS **NULL-TERMINATED (\0)** AUTOMATICALLY.

for(int i=0; msg[i]; i++){

- ITERATES THROUGH EACH CHARACTER IN THE STRING.
- → LOOP CONTINUES UNTIL IT HITS \0 (NULL CHARACTER).

KERNAL.C

video[i*2] = msg[i];

- WRITES THE CHARACTER TO VIDEO MEMORY.
- WHY i*2?
- EACH SCREEN CELL USES **2 BYTES**

video[i*2+1] = 0x07;

- SETS THE **ATTRIBUTE BYTE (COLOR)**.

MAKEFILE

```
all:
        nasm -f bin boot.asm -o myos.bin
```

WHAT IT DOES:

- ALL IS THE **DEFAULT TARGET**.
- WHEN YOU TYPE MAKE, THIS BLOCK RUNS.

MAKEFILE

nasm -f bin boot.asm -o myos.bin

- nasm → assembler (converts .asm → machine code)
- -f bin → output raw binary (needed for bootloader)
- boot.asm → input file
- -o myos.bin → output file

🎯 **RESULT:**

- CONVERTS YOUR ASSEMBLY BOOTLOADER INTO A **BOOTABLE BINARY FILE (MYOS.BIN)**

MAKEFILE

run:

```
qemu-system-i386 myos.bin
```

WHAT IT DOES:

- runs your os using an emulator.
- qemu-system-i386 → emulator for 32-bit x86 system
- myos.bin → your bootable os file
- 🎯 **RESULT:**
- BOOTS YOUR OS IN A VIRTUAL MACHINE
- YOU SHOULD SEE:

MAKEFILE

clean:

```
        rm -f myos.bin
```

WHAT IT DOES:

- DELETES THE GENERATED FILE.

COMMAND EXPLANATION:

- rm → REMOVE FILE
- -f → FORCE DELETE (NO ERROR IF FILE DOESN'T EXIST)
- 🎯 **RESULT:**
- KEEPS YOUR FOLDER CLEAN BY REMOVING COMPILED OUTPUT

OVERALL WORKFLOW

- **STEP-BY-STEP:**
- MAKE

  → Builds myos.bin
- MAKE RUN

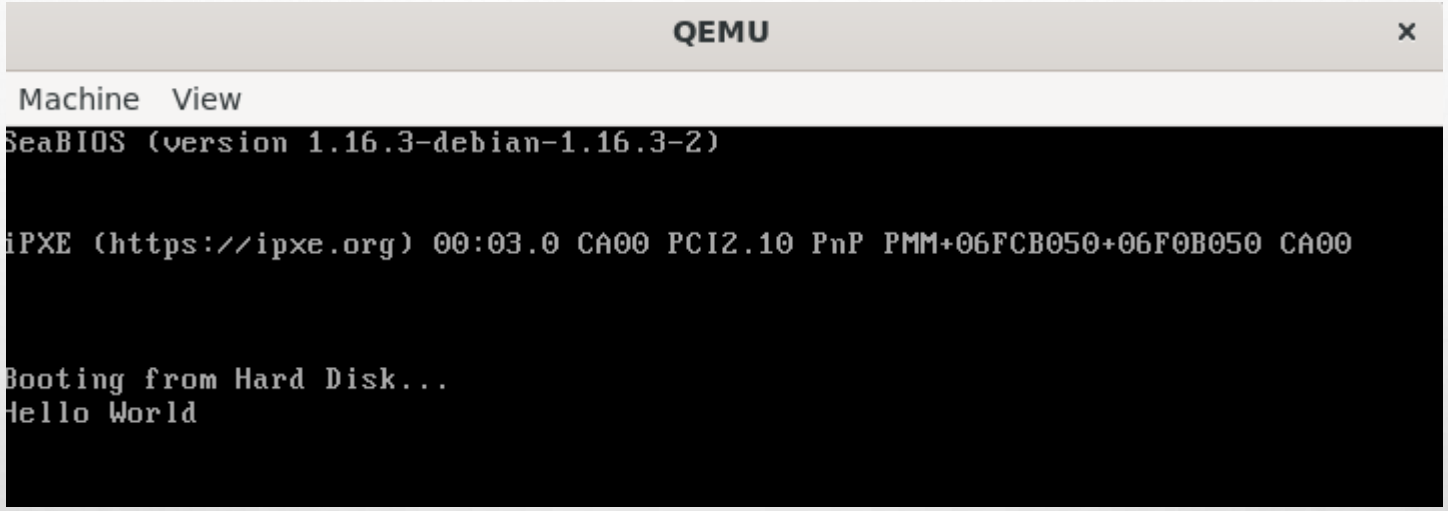
  → Runs OS in QEMU
- MAKE CLEAN

  → Deletes myos.bin

TERMINAL EXECUTION…

```
kanish@Kanishwar:~$ cd OS
kanish@Kanishwar:~/OS$ cd myos2
kanish@Kanishwar:~/OS/myos2$ make clean
rm -f myos.bin
kanish@Kanishwar:~/OS/myos2$ make
nasm -f bin boot.asm -o myos.bin
kanish@Kanishwar:~/OS/myos2$ make run
qemu-system-i386 myos.bin
```

FINAL RESULT



RESEARCH PAPERS REFERRED:

BARE BONES OS DEVELOPMENT GUIDE(OSDEV)

- HTTPS://WIKI.OSDEV.ORG/BARE_BONES?UTM_

MAKING AN OS FROM SCARCH (X86, VGA, BOOT PROCESS)

- HTTPS://MEDIUM.COM/%40RICARDITOMONTSERRAT/MAKING-AN-OS-FROM-SCRATCH-C8755BE9F4CC?UTM

THANK YOU!

- Boot.Asm by: Dharun and Vignesh
- Kernal.C by: Kanishwar and Vignesh
- Makefile by: all 3 (main execution and debugging involved checking all three files)
- GITHUB LINK: https://github.com/jkrsubzer0-0/OS_Project

BY:

- **DHARUN S V (241118)**
- **KANISHWAR S (241123)**
- **VIGNESH R S (241159)**